OPSFLEET

Retail Data Analysis Assistant

Logical Architecture

Component responsibilities and interaction flows

Production architecture and POC scope

Prepared for OpsFleet

November 2026

OpsFleet — Logical Architecture: Component-by-Component Explanation

Document type: High-Level Design (HLD) companion specification
Source: Supplied “Logical Architecture” PNG
Status: Proposed architecture; intended behavior, not a claim that every component has been implemented
Scope: Retail analytics assistant using LangGraph, Gemini and BigQuery

1. Purpose and reading guide

The diagram describes a conversational data-analysis system for retail executives. A user asks a question in natural language; the system authenticates the user, routes the request, retrieves trusted business context, generates and validates SQL, reads permitted BigQuery data, and formulates a grounded explanation. The same conversation can also manage saved analytical reports, subject to an explicit approval step before destructive actions.

The **rectangles** are application capabilities or processing steps. The **cylinders** represent persistent data stores or durable service-backed information. The arrows describe principal logical dependencies and control/data transfers, not necessarily one-to-one network requests or separate deployed services. LangGraph may host several of these capabilities as nodes within one Python process.

The following sections describe **all 17 labeled components** in the supplied diagram. For each, they cover its purpose, expected operation, interactions, and major controls.

2. Interface and entry controls

2.1 Executive CLI or web UI

Purpose. Provide the human-facing interface through which nontechnical executives ask questions, request comparisons and reports, inspect answers, and confirm or cancel operations.

How it works. The mandatory assignment prototype uses a command-line conversation. A production web client could provide formatted tables, charts, report downloads, and explicit confirmation buttons. Each request carries its conversation identifier and the user's question to the authenticated API. The interface displays analytical status, the final response, safe validation errors, and actionable clarification questions. If a report deletion requires approval, it presents the exact target set and captures an affirmative or negative decision.

Inputs: User questions, follow-up messages, approval decisions.
Outputs: Rendered answers, generated report previews, notices, confirmation prompts.
Controls: Never treat a client-supplied user ID, role, or list of authorized products as verified identity or entitlement. Do not expose hidden prompts, secrets, raw PII, or sensitive internal traces.

2.2 Auth and API boundary

Purpose. Separate untrusted client requests from trusted application services and establish the identity and authorization context for every interaction.

How it works. In production an API component (for example FastAPI on Cloud Run) validates authentication tokens, resolves the authenticated user and tenant, applies request limits, and loads the user's permissions and product entitlements from an authoritative source. It passes a trusted request context into LangGraph, rather than allowing the LLM to decide the user's privileges. The CLI prototype may use documented simulated users and entitlements, but must still demonstrate meaningful isolation.

Inputs: HTTP or CLI request, authenticated session/token, trusted entitlement records.
Outputs: Normalized request; verified identity, tenant, roles, allowed product scope, conversation ID.
Controls: Least privilege, deny-by-default access, input-size limits, rate limiting, authorization on report ownership, audit context. User identity remains server-controlled throughout the graph.

2.3 Versioned persona configuration

Purpose. Allow nondevelopers to change report style and tone without redeploying application code.

How it works. A governed configuration store holds versioned presentation instructions, for example “concise executive summary,” “formal board report,” or “prioritize action items.” The orchestrator loads an approved version when a run begins, optionally combining it with the current user's presentation preferences. The run records the version used, making changes auditable and reversible. Persona configuration influences wording, structure, and level of detail—not metric definitions, permitted tools, authorization, or security policies.

Inputs: Approved instruction version, user presentation preferences.
Outputs: Effective report/presentation configuration and configuration version ID.
Controls: Role-restricted editing, validation, approval and rollback; hard safety instructions are not editable through this mechanism.

3. Central orchestration and dispatch

3.1 LangGraph orchestrator

Purpose. Coordinate all steps of a stateful, multi-turn analytical conversation and report-management workflow.

How it works. LangGraph maintains a typed graph state, invokes individual nodes, evaluates conditional transitions, coordinates retries, and resumes after interruptions. A run may execute an analytical plan containing several SQL queries rather than one single query. The orchestrator interacts with durable checkpoints so an interrupted or failed conversation can continue later. It also emits telemetry for each node and tool invocation.

Representative state fields include: verified user context, thread/conversation ID, original question, resolved intent, current plan, selected schema/knowledge, SQL proposal, validation results, query references, sanitized analytical results, retry budget, report IDs, pending approval details, final answer, and trace ID.

Inputs: Verified request context, prior checkpoint, current persona version.
Outputs: Node actions, state updates, follow-up/approval prompts, final response.
Controls: Bounded iterations, model/tool timeouts, maximum query and token budgets, separate deterministic execution of sensitive actions. A graph node is not automatically a separate process, service, or autonomous agent.

3.2 Intent routing

Purpose. Determine which permitted workflow should handle the user's current message.

How it works. The router interprets the current message in conversational context and chooses a structured intent such as analytical query, database/schema question, create report, list/read saved report, delete report, clarification, or unsupported request. Classification may combine a fast LLM call with deterministic rules. It forwards each request to the analysis or saved-report-management branch and rejects unrelated or unauthorized operations.

Inputs: User message, trusted identity, relevant conversation state.
Outputs: Intent enum, normalized request, target workflow, optional clarification.
Controls: Classification is a usability mechanism, **not** the sole security barrier. Every downstream tool performs its own authorization checks. The diagram shows two main branches; schema questions and refusals can be subroutes within them or separate branches in implementation.

4. Analytical question-answering branch

4.1 Analysis workflow

Purpose. Manage the complete analytical investigation, including multi-step questions and evidence-based conclusions.

How it works. This LangGraph subworkflow resolves the user's question into measurable analytical tasks, obtains schema and business context, plans suitable SQL, executes validated queries, evaluates returned data, and hands the evidence to the synthesis step. Follow-up questions may reuse previous metric definitions or time periods, but must recheck the current user's access. A “why” question may need several queries—for example a revenue trend, order-count change, average order value, product-category decomposition, and return rates.

Inputs: Normalized question, verified authorization scope, conversation context.
Outputs: Analytical plan, execution results, evidence and limitations.
Controls: Distinguish correlation and descriptive decomposition from causality; stop or ask for clarification if the dataset cannot support the requested inference.

4.2 Retrieve schema metrics and Golden Trios

Purpose. Supply the model with the correct database structure, company metric definitions, and examples of high-quality historical analyses.

How it works. The retrieval component finds relevant table/column metadata, relationships, metric definitions, and Golden Trios: **Question → SQL Query → Analyst Report**. It may retrieve examples with semantic search and then rerank them using relevance, schema compatibility, business domain, quality, freshness, and access controls. Only a small, useful subset is added to the planning/generation context. This mechanism is organizational knowledge retrieval, not an instruction to blindly execute historical SQL.

Inputs: Current question, desired metrics, schema version, user authorization metadata.
Outputs: Relevant schema fragments, metric rules, vetted historical examples and provenance.
Controls: Apply access filters before retrieval results reach the model. Treat text retrieved from reports or database metadata as **untrusted data**, not privileged instructions. Historical definitions may be obsolete and must be version-checked.

4.3 Golden Knowledge index

Purpose. Make historical analyst reasoning findable at question time, rather than forcing the assistant to infer every analytical pattern from raw database tables.

How it works. The index stores searchable representations of approved Golden Trios, typically embeddings plus metadata and references to authoritative source objects (for example in GCS). A retrieval request selects similar historical questions/reports. Production ingestion validates, classifies, versions, and—where required—requires expert approval before publishing new examples and refreshing the index. The index is distinct from the authoritative source-document repository.

Inputs: Approved Trio documents, embeddings, metadata, access classifications, indexing updates.
Outputs: Ranked Trio identifiers, similarity scores, metadata and source references.
Controls: Tenant/product isolation, provenance, approval status, lifecycle management; prevent automatic ingestion of unchecked agent outputs, which could reinforce mistakes.

4.4 SQL planning and generation

Purpose. Translate an executive's analytical objective into one or more executable BigQuery queries.

How it works. The planner decomposes the problem into well-defined subquestions, chooses correct joins and metrics, and generates GoogleSQL statements using Gemini. Context includes the current question, restricted schema, semantic metric definitions, and approved Golden examples. A structured response should identify the analytical objective, metric, dimensions, filters, date window, SQL text, and assumptions. For multi-step analysis, later queries may depend on earlier sanitized results.

For example, a decline in revenue can be examined through the relationship **revenue** = **qualifying order count** × **average order value**, with additional decomposition by region, category, customer segment, or returns as supported by the data.

Inputs: Plan, schema/metric context, Golden examples, authorized product scope (as guidance only).
Outputs: Candidate SQL and structured analysis-step metadata.
Controls: Generated SQL is **untrusted until validated**. Avoid unrestricted schema discovery, unrestricted model tool calls, excessive context, or unsupported metrics. SQL repair must return through validation.

4.5 SQL and access validator

Purpose. Enforce the non-negotiable boundary between LLM-generated SQL and the analytical database.

How it works. A deterministic validator parses each query with a BigQuery-aware SQL parser and checks its abstract syntax tree, statement type, table references, columns, functions, joins, product-scope restrictions, result granularity, and cost parameters. It allows only the intended read-only analytics. It can perform a BigQuery dry run to catch syntax/schema problems and estimate processing cost, then either approve the query, reject it, request clarification, or send a correctable error back into the bounded generation loop.

Inputs: Candidate SQL, verified user entitlements, approved schema/metric policy, resource budgets.
Outputs: Validated SQL or typed rejection with safe diagnostic metadata.
Controls: Deny writes, scripts, restricted identifiers/PII, unsafe functions and references; enforce result limits and cost/time budgets. A regex check or model-based “safety judgment” alone is inadequate. The backend's actual BigQuery permissions and secure views must independently enforce least privilege.

4.6 BigQuery secure data access

Purpose. Execute permitted analytics against the retail transaction dataset while containing sensitive data exposure.

How it works. A trusted query runner submits validated SQL through a read-only BigQuery identity or suitable governed access service, handles asynchronous job status, applies bytes-billed/time/result limits, and returns typed, bounded query results. The assignment tables are bigquery-public-data.thelook_ecommerce.orders, order_items, products, and users. The users table contains direct identifiers; analytical queries should ordinarily return authorized aggregates rather than raw customer records. Product-level entitlement mapping must be defined by the application because the public dataset does not define executives' permitted products.

Inputs: Validated query, execution limits, trusted authorization context.
Outputs: Sanitized and/or privacy-approved tabular results, schema, row counts, job ID, processing

metrics, errors.
Controls: Secure views/column controls, product authorization, aggregate privacy thresholds, suppression of re-identifiable cohorts, limits on columns/rows and query cost. If a single service account queries on behalf of many users, its IAM identity alone does not enforce individual executives' scopes. Exclude raw PII before LLM analysis whenever possible.

4.7 Evidence-based synthesis

Purpose. Convert approved query results into clear, accurate, decision-useful executive explanations and reports.

How it works. The synthesizer receives bounded, privacy-safe results and the original analytical questions. It constructs an answer containing the metric definitions, period, comparison, observed changes, supporting numbers, uncertainty, and possible action items. It can produce a short response or a structured report (executive summary, findings, evidence, caveats, recommendations) under the active persona configuration. Before publication, an output checker inspects factual grounding, unauthorized data, and PII. The synthesized answer then returns through the LangGraph orchestrator to the client.

Inputs: Question, query results, evidence provenance, metric semantics, persona preferences.
Outputs: Final answer or report, evidence references, limitation statements.
Controls: Never fabricate numerical values, infer causation merely from correlation, or reveal prohibited PII. Separate **observation**, **possible explanation**, and **recommended action**. Explain empty or insufficient datasets honestly.

5. Report lifecycle branch

5.1 Saved report management

Purpose. Provide a controlled library of reports created by the assistant and belonging to authorized users.

How it works. A report-management component supports creation, listing, retrieval, metadata search, and deletion requests. Each report has a stable ID, owner/tenant, creation time, conversation association, content or artifact reference, and lifecycle status. Report storage may be PostgreSQL with larger exported files in GCS. The component resolves requests such as “delete every report mentioning Client X” into explicit report IDs that the authenticated user actually owns. Deletion requests are passed to the approval gate; creation and reading generally are not destructive.

Inputs: Report operation, verified user, report criteria, report content/metadata.
Outputs: Saved report, matching report list, pending deletion proposal, or status.
Controls: Ownership checks on **every** operation, safe search, scoped result list, idempotent storage actions, audited lifecycle changes. A report's text is not a source of executable instructions.

5.2 Human confirmation gate

Purpose. Prevent accidental or malicious destructive operations without requiring the user to restart the conversation.

How it works. The application first resolves the exact set of user-owned reports, presents their names and count, and stores a pending action with stable IDs and a confirmation token/operation ID. LangGraph can issue a durable interrupt and resume the same conversation after an explicit approve/cancel response. On approval, the system revalidates ownership and the frozen targets, performs the idempotent deletion operation, and records its outcome. A cancellation preserves reports. A recoverable soft delete is a reasonable initial design if its retention behavior is disclosed.

Inputs: Pending operation, exact report IDs, verified confirmation/cancellation.
Outputs: Authorized execution decision, cancellation or expired-action outcome, audit event.
Controls: Approval applies only to the displayed target set; never treat a generic "yes" from another thread as authorization. Since an interrupted node can restart on resume, put side effects behind idempotent action IDs and appropriate transactional checks. The shown arrow from gate to storage abbreviates the confirm–revalidate–execute sequence.

5.3 Saved reports

Purpose. Durable repository for the executive reports generated and retained by the system.

How it works. The store contains report metadata, ownership, report content or references to exported objects, creation time, conversation ID, and deletion state. It supports targeted retrieval and transactional updates. In the proposed production design, Cloud SQL PostgreSQL provides structured records while GCS can hold large PDF/CSV exports. A retention and restoration policy defines what soft-deleted reports mean and when associated objects are permanently removed.

Inputs: New reports, metadata, lifecycle transitions, approved deletion operations.
Outputs: Authorized report records and persisted operation status.
Controls: Tenant/user isolation, encryption, retention, backups, auditability, storage of privacy-approved content only. This is **not** the read-only BigQuery analytical database.

6. Cross-cutting state and operations

6.1 Conversation checkpoints

Purpose. Preserve execution state across turns, transient failures, and human approval interruptions.

How it works. LangGraph writes durable checkpoints keyed by thread/conversation ID. A resumed invocation restores relevant messages, working state, active plan, budget counters, and pending interrupt. A production PostgreSQL-backed checkpointer is suitable; a local checkpointer can support the prototype. Conversation memory is distinct from long-term user preferences and the Golden Knowledge repository. Large raw result sets should not be copied wholesale into checkpoints; store minimized, sanitized summaries or stable references.

Inputs: Graph state transitions and thread IDs.
Outputs: Restored state and pause/resume continuity.
Controls: Access-controlled checkpoints, privacy-aware retention, no credentials or unnecessary raw PII, careful compatibility across graph-state schema changes.

6.2 Traces and metrics

Purpose. Make production behavior observable, measurable, debuggable, and auditable.

How it works. The orchestrator and its tools emit correlated events with trace/run IDs, node names, model/version, prompt/config version, timing, token usage, SQL validation outcomes, BigQuery job IDs and bytes processed, retry counts, errors, and approval outcomes. Operational monitoring aggregates reliability, quality, security, latency and cost metrics; detailed spans support investigation of individual failures. LangSmith can provide agent-level traces while Cloud Logging/Monitoring cover infrastructure and service telemetry.

Inputs: Events from graph nodes, model calls, BigQuery jobs, report actions and security decisions.
Outputs: Traces, dashboards, alerts, evaluation datasets and audit evidence.
Controls: Redact sensitive prompts/results, restrict telemetry access, configure retention, and avoid logging secrets or prohibited customer data. A trace should explain failures without becoming a second uncontrolled copy of the underlying dataset.

7. End-to-end examples

7.1 Analytical request

Question: “Why did revenue drop in California last quarter compared with Texas?”

The client sends the request; the API validates the executive's identity and product entitlements.

LangGraph restores the conversation and loads the current approved persona version.

Intent routing selects the analysis branch.

Retrieval provides relevant table relationships, revenue definitions, and any authorized Golden Trios.

Planning proposes a series of comparisons: revenue, order counts, average order value, category composition and return status where available.

Every SQL statement is validated for read-only behavior, correct data access, privacy, and budget.

BigQuery returns permitted aggregates and query metadata.

The synthesizer explains the measured differences, distinguishes hypotheses from proven explanations, and suggests evidence-grounded follow-up actions.

LangGraph returns the answer, persists appropriate state, and emits traces and metrics.

7.2 Destructive report request

Question: “Delete all reports we made in this conversation.”

The API authenticates the user and restores the conversation ID.

Intent routing selects saved report management.

Report management queries only user-owned reports associated with the current conversation.

The system lists the exact target reports and requests explicit confirmation.

LangGraph checkpoints the pending action and pauses.

Cancel means no deletion. Confirm triggers revalidation, idempotent execution, and audit logging.

The user receives a precise status indicating the affected reports.

8. Clarifications and refinements to the supplied diagram

The source image is a sound **logical** overview, but its arrows intentionally omit some important runtime behavior:

Retrieval feeds SQL planning. The diagram shows “Retrieve schema metrics and Golden Trios” and “SQL planning and generation” as two branches from analysis. In practice, retrieved context normally flows **into** planning/generation, even if schema discovery and other preparation can run concurrently.

Knowledge index direction is conceptual. The retrieval component queries the index and receives results; the arrow should be considered bidirectional information exchange, not ingestion-only.

The analysis can loop. SQL errors and some result-quality issues lead to a bounded repair/replanning cycle through the validator; the image depicts only the successful path.

Evidence-based synthesis returns to the user. The return arrow to LangGraph is shown; the subsequent response to the API/client is implicit.

Report creation is also supported. Although the diagram highlights the deletion confirmation gate, ordinary report creation and retrieval should go to the report store without destructive confirmation. The report-generation output of synthesis may be saved via report management.

The confirmation gate does not delete by itself. Approval resumes a trusted executor that rechecks ownership and targets; the image abstracts this important boundary.

Authentication is cross-cutting. Every query and report action must enforce authorization, even after the initial API check. The BigQuery secure-access path needs explicit product-scope enforcement.

Several missing boxes are intentionally logical subfunctions. Output PII inspection, error classification, retries, user preference storage, semantic metric governance, Golden Trio ingestion

and offline evaluation may be drawn as additional nodes in a more detailed HLD. Their absence from this image does not mean they are unnecessary.

External systems are not deployment boundaries here. No individual rectangle necessarily represents a separately deployed Cloud Run service. The companion deployment diagram should map software functions onto Cloud Run, Cloud SQL, GCS, Vertex AI and BigQuery.

9. Prototype versus full production design

CLI chat, BigQuery SQL, multi-step analysis and reports

Mandatory prototype: Working

Production HLD: Fully designed

PII safety and per-user product access

Mandatory prototype: Working demonstration

Production HLD: Defense in depth and IAM/data governance

Saved report deletion with explicit approval

Mandatory prototype: Working

Production HLD: Durable, audited lifecycle and retention

SQL repair, outage handling and bounded retries

Mandatory prototype: Working

Production HLD: SLOs, alerting and fallback policies

Agent traces and metrics

Mandatory prototype: Working

Production HLD: Distributed observability and dashboards

Golden Trio retrieval and curation

Mandatory prototype: Optional implementation

Production HLD: Fully designed

User/system learning and prompt persona management

Mandatory prototype: Optional implementation

Production HLD: Fully designed

Charts, email, web search and advanced features

Mandatory prototype: Extensibility hooks only

Production HLD: Integration design

10. Design decisions to confirm with OpsFleet

Definition of gross/net revenue and handling of cancellations and returns.

Definition and eligibility window for churn, which is not a direct source-table field.

Mapping from authenticated executives to permitted product IDs/categories.

Minimum aggregation/cohort privacy thresholds and permitted customer-level analyses.

Whether illustrative synthetic Golden Trios are acceptable.

Whether deletion is soft or permanent, with retention expectations.

Supported authentication arrangement for the CLI prototype and acceptable local persistence.

Summary: The diagram separates conversational orchestration from analytical execution, organizational knowledge, report lifecycle, and operational oversight. Its most important design feature is the trust boundary: Gemini and LangGraph support flexible analysis, while deterministic code and managed data permissions govern exactly what can be queried, disclosed, stored, or deleted.